MBU
MOHAN BABU UNIVERSITY
DREAM. BELIEVE. ACHIEVE

PROJECT DOCUMENTATION

FUEL CONSUMPTION ANALYSIS SYSTEM

A beginner-level application for recording fuel purchases, tracking distance travelled, and analysing vehicle mileage.

Submitted by

Team Member	Roll Number
G. Akash	25102A010752
Y. Gayatri	25102A010736
Y. Harsha Vardhan	25102A010738
V. Vishnu Priya	25102A010740
G. Karthik	25102A010741

Mohan Babu University

Programme / Department: AIML

Academic Project Report

1. INDEX

This document presents the purpose, design, implementation plan, and evaluation approach for the Fuel Consumption Analysis System.

No.	Section	Description
1	Index	Quick guide to the report
2	Title Page	Project and team details
3	Acknowledgement	Thanks and support received
4	Abstract	Project summary
5	Table of Contents	Detailed section list
6	Introduction	Background and motivation
7	Problem Statement	Problem addressed by the system
8	Objective and Scope	Goals, boundaries, and limitations
9	Technology Stack and Requirements	Software, hardware, and functional needs
10	System Architecture and Design	Modules and data flow
11	Database Design	Tables, keys, and relationships
12	Implementation	Core logic and calculations
13	Testing and Result	Test cases and expected outcomes
14	Conclusion and Future Enhancements	Summary and possible improvements
15	References	Resources used for the report

2. TITLE PAGE

Project Title: Fuel Consumption Analysis System

Institution: Mohan Babu University

Programme / Department: AIML

Project Type: Beginner-level software application

Team Members:

• G. Akash — 25102A010752

• Y. Gayatri — 25102A010736

• Y. Harsha Vardhan — 25102A010738

• V. Vishnu Priya — 25102A010740

• G. Karthik — 25102A010741

The project is designed to demonstrate how vehicle fuel records can be stored and analysed to calculate mileage and fuel cost per kilometre.

3. ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Mohan Babu University and the AIML department for providing us with the opportunity to work on the project titled “Fuel Consumption Analysis System.”

We thank our faculty members and project guide for their guidance, encouragement, and valuable suggestions during the planning and documentation of this project. Their support helped us understand how a real-world problem can be translated into a software solution.

We also thank our team members for their cooperation, discussions, and contributions to the project. Finally, we acknowledge the authors and maintainers of the technical documentation and learning resources referred to while preparing this report.

4. ABSTRACT

The Fuel Consumption Analysis System is a simple software application intended to help users maintain vehicle details and fuel purchase records. For each fuel entry, the system records the vehicle, date, odometer reading, quantity of fuel purchased in litres, and total cost. Using valid odometer readings, it estimates the distance travelled between refuelling events and calculates fuel efficiency in kilometres per litre (km/L). It can also calculate fuel cost per kilometre when the required data is available.

The system distinguishes between full-tank mileage calculations and approximate distance-per-litre estimates. A full-tank calculation is more reliable when the tank is filled to a consistent level at the start and end of the measurement period. If this condition is not met, the result should be treated as an approximation. The system validates records to prevent negative quantities, invalid costs, and decreasing odometer readings.

The proposed implementation uses Java for application logic and MySQL for persistent storage. The project is suitable for beginners because it introduces data entry, validation, database operations, arithmetic calculations, and report generation in a practical context.

5. TABLE OF CONTENTS

The report is organised into the following sections:

1	Index
2	Title Page
3	Acknowledgement
4	Abstract
5	Table of Contents
6	Introduction of the Project
7	Problem Statement
8	Objective and Scope
9	Technology Stack and Requirements
10	System Architecture and Design
11	Database Design
12	Implementation
13	Testing and Result
14	Conclusion and Future Enhancements
15	References

6. INTRODUCTION OF THE PROJECT

Fuel is a recurring expense for vehicle owners, and understanding fuel efficiency can help with budgeting, vehicle maintenance, and driving decisions. When fuel purchases are recorded only on paper or remembered informally, it can be difficult to compare mileage across different dates or identify changes in fuel consumption.

The Fuel Consumption Analysis System provides a structured way to maintain these records. It stores vehicle information and fuel entries, checks whether the entered data is valid, and calculates useful measures such as distance travelled, mileage, total fuel expenditure, and fuel cost per kilometre.

The project focuses on a small, understandable workflow rather than a complex fleet-management platform. A user can register a vehicle, add fuel purchases, view previous entries, and generate a summary for a selected vehicle. Sample records can be used to demonstrate the calculations.

Key terms

- **Odometer reading:** cumulative distance shown by a vehicle's odometer, usually in kilometres.
- **Distance travelled:** difference between two valid odometer readings.
- **Mileage / fuel efficiency:** distance travelled divided by fuel consumed, expressed in km/L.
- **Fuel cost per kilometre:** applicable fuel cost divided by the distance travelled.

7. PROBLEM STATEMENT

Vehicle users may not have a consistent method to record refuelling events and calculate mileage. Manual calculations are time-consuming and can lead to errors, especially when multiple vehicles or many fuel entries are involved. Records may also be incomplete, making it difficult to compare fuel efficiency over time.

The problem is to develop a basic system that stores vehicle and fuel purchase details, validates the entered information, calculates distance travelled between suitable odometer readings, and reports mileage and fuel expenses wherever sufficient data is available.

The system should avoid presenting misleading calculations. For example, mileage should not be calculated from a single fuel entry without a valid distance interval and a reasonable estimate of fuel consumed. Full-tank-to-full-tank entries should be identified separately from approximate distance-per-litre calculations.

Expected outcome

• A central record of vehicles and fuel purchases.

• Automatic calculation of distance between valid odometer readings.

• Mileage and fuel cost summaries with clear units.

• Validation messages for incomplete or inconsistent records.

8. OBJECTIVE AND SCOPE

8.1 Objectives

• Register and maintain basic vehicle information.

• Record date, odometer reading, litres purchased, and total fuel cost for each refuelling event.

• Calculate distance travelled using valid consecutive odometer readings for the same vehicle.

• Calculate mileage in km/L when the fuel amount for the measured interval is known.

• Calculate fuel cost per kilometre when corresponding cost and distance are available.

• Compare fuel efficiency across records and show simple summaries.

• Validate inputs and flag data that should not be used in calculations.

8.2 Scope

The proposed system covers sample vehicles and fuel records for a small-scale demonstration. It can support adding, viewing, updating, and deleting records, along with basic summary reports. The initial version is intended for a single user and can be run locally on a computer.

8.3 Boundaries and assumptions

• Distance is recorded in kilometres and fuel quantity in litres.

• Odometer readings must belong to the same vehicle and must not decrease across valid records.

• A full-tank mileage result should use a consistent full-tank-to-full-tank method.

• If a record is not a full-tank measurement, any estimate must be labelled approximate and its assumptions explained.

• The application does not connect to vehicle sensors, fuel stations, or live GPS services.

9. TECHNOLOGY STACK AND REQUIREMENTS

9.1 Proposed technology stack

Component	Technology	Purpose
Programming language	Java	Application logic, input handling, calculations, validation
Database	MySQL	Persistent storage for vehicles and fuel records
Database connectivity	JDBC	Communication between Java and MySQL
Development environment	IntelliJ IDEA / Eclipse / VS Code	Writing and running the application
Database client	MySQL Workbench or command line	Creating tables and inspecting records
Interface	Console-based menu (initial version)	Simple user interaction

9.2 Hardware requirements

• Computer or laptop with a modern processor.

• At least 4 GB RAM recommended.

• Sufficient disk space for the JDK, IDE, MySQL, and project files.

9.3 Software requirements

• Java Development Kit (JDK), preferably a currently supported version.

• MySQL Server and a database client.

• An IDE or text editor capable of compiling Java.

• JDBC driver for the selected MySQL version.

9.4 Functional requirements

• Add and manage vehicle details.

• Add and view fuel purchase records.

• Validate required fields and numeric values.

• Calculate mileage and fuel cost per kilometre when valid data exists.

• Display vehicle-wise fuel history and summaries.

9.5 Non-functional requirements

• Usability: menus and error messages should be clear.

• Reliability: calculations should use validated records.

• Data integrity: database relationships should prevent orphan fuel records.

• Maintainability: calculation logic and database code should be separated.

10. SYSTEM ARCHITECTURE AND DESIGN

10.1 Architecture overview

The application follows a simple three-layer structure. The user interface collects input and displays results. The application layer validates data and performs calculations. The data layer stores and retrieves vehicle and fuel records from MySQL.

USER / CONSOLE MENU
Add vehicle • Enter fuel record • View history • View analysis
↓
APPLICATION LAYER — JAVA
Input validation • Mileage calculations • Cost calculations • Report formatting
↓ JDBC
DATABASE LAYER — MYSQL
vehicles table • fuel_records table

10.2 Main modules

• **Vehicle Management:** add, view, update, and remove vehicle information.

• **Fuel Record Management:** save fuel purchase date, odometer reading, litres, and cost.

• **Validation:** check required fields, positive litres and cost, and consistent readings.

• **Analysis:** calculate distance, mileage, and cost per kilometre for eligible records.

• **Reporting:** show fuel history and simple vehicle-wise summaries.

10.3 Data flow

The user selects an operation and enters data. The application validates the values before saving them. When analysis is requested, records are retrieved in date/odometer order for the selected vehicle. The system calculates distance only for valid consecutive readings and displays mileage only when the relevant fuel-consumption interval can be established.

11. DATABASE DESIGN

The proposed database contains two core tables: **vehicles** and **fuel_records**. Each vehicle can have many fuel records, while each fuel record belongs to exactly one vehicle.

11.1 Table: vehicles

Field	Type	Constraint / Description
vehicle_id	INT	Primary key; auto-increment
vehicle_name	VARCHAR(80)	Required; user-friendly name
registration_number	VARCHAR(20)	Unique vehicle registration identifier
fuel_type	VARCHAR(20)	Petrol, diesel, CNG, etc.
created_at	TIMESTAMP	Record creation time

11.2 Table: fuel_records

Field	Type	Constraint / Description
record_id	INT	Primary key; auto-increment
vehicle_id	INT	Foreign key referencing vehicles.vehicle_id
fuel_date	DATE	Date of purchase
odometer_km	DECIMAL(10,1)	Cumulative odometer reading in km
litres	DECIMAL(8,3)	Fuel purchased, greater than zero
total_cost	DECIMAL(10,2)	Total purchase cost, greater than zero
is_full_tank	BOOLEAN	Whether the tank was filled to full
notes	VARCHAR(255)	Optional notes

11.3 Relationship

vehicles (1) ■■■■■■■■■ (many) fuel_records. The foreign key vehicle_id connects each fuel record to its vehicle. Deleting a vehicle should be handled carefully; either prevent deletion while records exist or use a deliberate cascade rule.

11.4 Example SQL schema

CREATE TABLE vehicles (vehicle_id INT PRIMARY KEY AUTO_INCREMENT, vehicle_name VARCHAR(80) NOT NULL, registration_number VARCHAR(20) UNIQUE, fuel_type VARCHAR(20) NOT NULL, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP); CREATE TABLE fuel_records (record_id INT PRIMARY KEY AUTO_INCREMENT, vehicle_id INT NOT NULL, fuel_date DATE NOT NULL, odometer_km DECIMAL(10,1) NOT NULL, litres DECIMAL(8,3) NOT NULL, total_cost DECIMAL(10,2) NOT NULL, is_full_tank BOOLEAN NOT NULL DEFAULT FALSE, notes VARCHAR(255), FOREIGN KEY (vehicle_id) REFERENCES vehicles(vehicle_id), CHECK (odometer_km >= 0), CHECK (litres > 0), CHECK (total_cost > 0));

12. IMPLEMENTATION

12.1 Application workflow

• Start the application and connect to the MySQL database.

• Display a menu: add vehicle, add fuel record, view records, analyse mileage, view summary, or exit.

• For a new fuel record, select the vehicle and enter date, odometer reading, litres, total cost, and full-tank status.

• Validate the entry before storing it. Reject invalid numeric values and flag odometer readings lower than the previous valid reading for the same vehicle.

• When the user requests analysis, retrieve the vehicle's records in chronological order and calculate only metrics supported by the available data.

12.2 Core formulas

Distance travelled (km) = Current odometer reading – Previous odometer reading

Mileage (km/L) = Distance travelled (km) ÷ Fuel consumed (L)

Fuel cost per kilometre (■/km) = Fuel cost applicable to the measured interval (■) ÷ Distance travelled (km)

Important: the litres purchased at a refuelling event are not automatically equal to the litres consumed over the previous interval. A reliable full-tank method uses a full tank at the start and refills to full at the end; the litres added at the end then approximate the fuel consumed over that interval. For partial fills or inconsistent tank levels, the application should show an approximate result only if its assumptions are explicit.

12.3 Example calculation

Item	Sample value
Previous full-tank odometer	12,000 km
Current full-tank odometer	12,300 km
Fuel added to refill to full	15 L
Cost of refill	■1,650
Distance travelled	12,300 – 12,000 = 300 km
Estimated mileage	300 ÷ 15 = 20 km/L
Approx. fuel cost per km	1,650 ÷ 300 = ■5.50/km

This example assumes the vehicle was filled to full at both measurement points and the 15 litres added at the second stop represents the fuel used during the 300 km interval.

12.4 Pseudocode

FOR each vehicle: sort valid fuel records by odometer reading FOR each pair of consecutive records: distance = current.odometer - previous.odometer IF distance <= 0: flag record; do not calculate ELSE IF records support a full-tank interval: fuel_used = litres added at ending full-tank refill mileage = distance / fuel_used cost_per_km = ending_refill_cost / distance display result with units ELSE: display "insufficient data" or a clearly labelled estimate

13. TESTING AND RESULT

Testing checks whether the application accepts valid information, rejects invalid entries, and produces correct calculations. The following test cases can be executed with sample data during project demonstration.

Test ID	Scenario / Input	Expected result
TC01	Add a vehicle with valid details	Vehicle is saved and assigned an ID
TC02	Add a valid fuel record	Record is saved under the selected vehicle
TC03	Enter zero or negative litres	Validation error; record is not saved
TC04	Enter a negative fuel cost	Validation error; record is not saved
TC05	Enter an odometer lower than the previous reading	Record is flagged or rejected for analysis
TC06	Calculate 300 km using 15 L	Mileage result is 20 km/L
TC07	Request mileage with only one record	System reports insufficient data
TC08	Partial-fill record without valid interval	No definitive mileage; limitation is explained
TC09	View history for selected vehicle	Only that vehicle's records are shown in order
TC10	Database connection unavailable	Useful error is displayed; app does not fail silently

13.1 Result

With the proposed design and sample data, the system is expected to store vehicle and refuelling information, validate key inputs, and calculate distance, mileage, and fuel cost per kilometre for eligible intervals. For the sample full-tank interval, a distance of 300 km and 15 litres of fuel gives an estimated mileage of 20 km/L and a fuel cost of ■5.50/km when the refill costs ■1,650.

The final implementation should record the actual test outcomes after the program is executed. This report describes planned tests and expected results; it does not claim that the application has already been run or independently verified.

14. CONCLUSION AND FUTURE ENHANCEMENTS

14.1 Conclusion

The Fuel Consumption Analysis System provides a straightforward way to organise vehicle fuel purchases and analyse fuel efficiency. By maintaining structured records and applying validation rules, it reduces manual calculation effort and makes fuel expenses easier to review. The project demonstrates core software development concepts such as modular design, database relationships, input validation, arithmetic processing, and testing.

A key design principle is to report mileage only when the underlying records support a meaningful calculation. Full-tank-to-full-tank records provide a stronger estimate of fuel consumed over a distance interval than isolated or partial fuel purchases. Communicating these limitations makes the results more useful and trustworthy.

14.2 Future enhancements

- Develop a graphical user interface or web-based dashboard.
- Add charts showing mileage and monthly fuel expenditure.
- Export vehicle summaries to PDF or CSV.
- Support multiple users with login and role-based access.
- Add filters by date range, vehicle, and fuel type.
- Include alerts when mileage drops below a user-defined threshold.
- Add backup and restore features for the database.
- Allow optional odometer and fuel receipt attachments.
- Introduce more advanced analytics after sufficient reliable records are collected.

15. REFERENCES

The following official documentation and learning resources are relevant to the proposed technology stack and design. Access dates may be added by the team when the final implementation is completed.

1. Oracle. *Java Documentation.* https://docs.oracle.com/en/java/

2. Oracle. *JDBC Basics.* https://docs.oracle.com/javase/tutorial/jdbc/basics/

3. MySQL. *MySQL 8.4 Reference Manual.* https://dev.mysql.com/doc/refman/8.4/en/

4. MySQL. *MySQL Connector/J Developer Guide.* https://dev.mysql.com/doc/connector-j/en/

5. Sommerville, Ian. *Software Engineering.* Pearson. (For general software engineering concepts, requirements, design, and testing.)

— End of Project Documentation —